THE NETWORK LAYER

- The network layer is concerned with getting packets from the source all the way to the destination.
- The network layer must know about the topology of the network (i.e., the set of all routers and links) and choose appropriate paths through it, even for large networks.
- It must also take care when choosing routes to avoid overloading some of the communication lines and routers while leaving others idle.
- **Network Layer Design Issues**
- **Store-and-Forward Packet Switching**
- The major components of the network are the ISP's equipment shown inside the shaded oval, and the customers' equipment, shown outside the oval.
- Host H1 is directly connected to one of the ISP's routers, A, perhaps as a home computer that is plugged into a DSL modem.

- A host with a packet to send transmits it to the nearest router, either on its own LAN or over a point-to-point link to the ISP.
- The packet is stored there until it has fully arrived and the link has finished its processing by verifying the checksum.
- Then it is forwarded to the next router along the path until it reaches the destination host, where it is delivered.

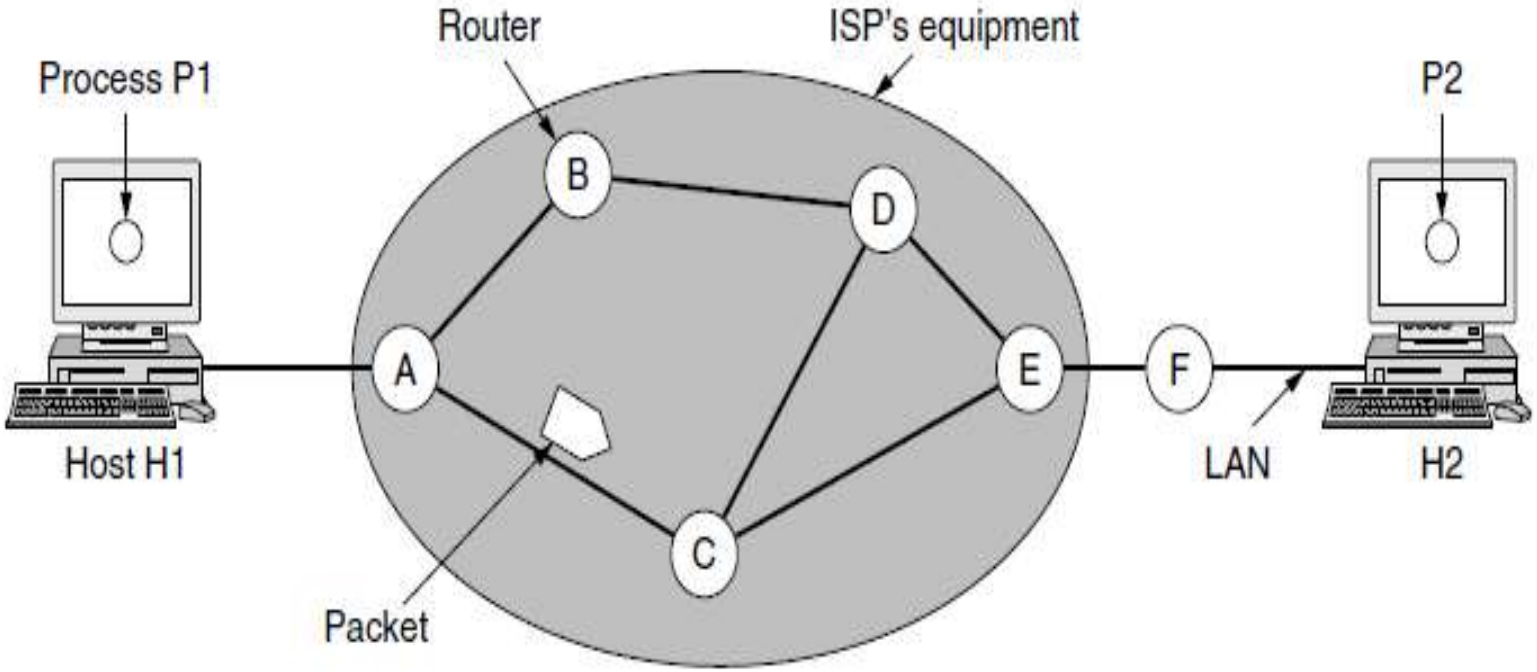
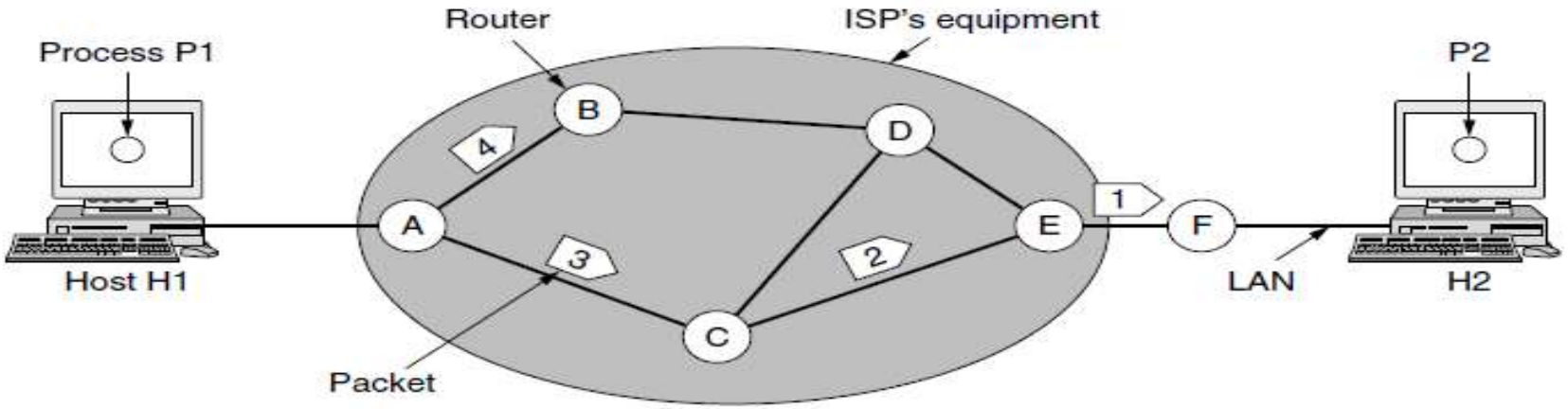


Figure 5-1. The environment of the network layer protocols.

- **Services Provided to the Transport Layer**
- The network layer provides services to the transport layer at the network layer/transport layer interface.
- The services need to be carefully designed with the following goals in mind:
- 1. The services should be independent of the router technology.
- 2. The transport layer should be shielded from the number, type, and topology of the routers present.
- 3. The network addresses made available to the transport layer should use a uniform numbering plan, even across LANs and WANs.
- Network service should be connectionless, with primitives SEND PACKET and RECEIVE PACKET and little else.
- In particular, no packet ordering and flow control should be done, because the hosts are going to do that anyway and there is usually little to be gained by doing it twice.
- This reasoning is an example of the end-to-end argument, a design principle that has been very influential in shaping the Internet .
- Furthermore, each packet must carry the full destination address, because each packet sent is carried independently of its predecessors, if any.

- **Implementation of Connectionless Service**
- Having looked at the two classes of service the network layer can provide to its users, it is time to see how this layer works inside.
- Two different organizations are possible, depending on the type of service offered. If connectionless service is offered, packets are injected into the network individually and routed independently of each other.
- No advance setup is needed. In this context, the packets are
- frequently called datagrams and the network is called datagram network.
- If connection-oriented service is used, a path from the source router all the way to the destination router must be established before any
- data packets can be sent.
- This connection is called a VC (virtual circuit), in analogy the physical circuits set up by the telephone system, and the network is called a virtual-circuit network.
- Suppose that the process P1 in Fig. has a long message for P2. It hands the message to the transport layer, with instructions to deliver it to process P2 on host H2.

- At A, packets 1, 2, and 3 are stored briefly, having arrived on the incoming link and had their checksums verified.
- Then each packet is forwarded according to A's table, onto the outgoing link to C within a new frame.
- Packet 1 is then forwarded to E and then to F. When it gets to F, it is sent within a frame over the LAN to H2. Packets 2 and 3 follow the same route.



A's table (initially)

Dest.	Line
A	–
B	B
C	C
D	B
E	C
F	C

A's table (later)

A	–
B	B
C	C
D	B
E	B
F	B

C's table

A	A
B	A
C	–
D	E
E	E
F	E

E's table

A	C
B	D
C	C
D	D
E	–
F	F

Figure 5-2. Routing within a datagram network.

- **Implementation of Connection-Oriented Service**
- Instead, when a connection is established, a route from the source machine to the destination machine is chosen as part of the connection setup and stored in tables inside the routers.
- That route is used for all traffic flowing over the connection, exactly the same way that the telephone system works.
- When the connection is released, the virtual circuit is also terminated.
- With connection-oriented service, each packet carries an identifier telling which virtual circuit it belongs to.
- Here, host H1 has established connection 1 with host H2. This connection is remembered as the first entry in each of the routing tables.
- The first line of A's table says that if a packet bearing connection identifier 1 comes in from H1, it is to be sent to router C and
- given connection identifier 1.
- Similarly, the first entry at C routes the packet to E, also with connection identifier 1.

- Now let us consider what happens if H3 also wants to establish a connection to H2. It chooses connection identifier 1 (because it is initiating the connection and this is its only connection) and tells the network to establish the virtual circuit.
- This leads to the second row in the tables

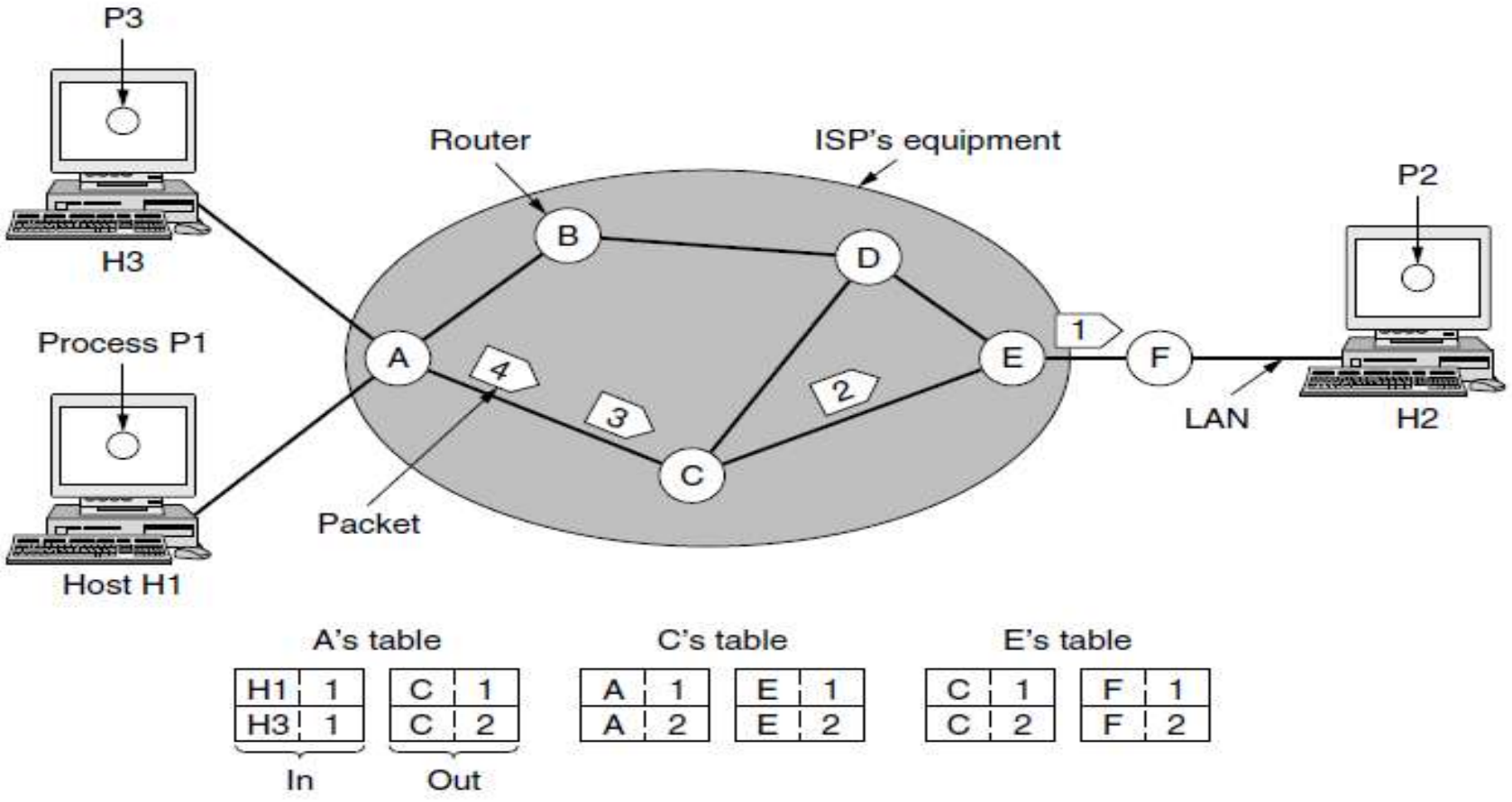


Figure 5-3. Routing within a virtual-circuit network.

- **Comparison of Virtual-Circuit and Datagram Networks**

Issue	Datagram network	Virtual-circuit network
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC

Figure 5-4. Comparison of datagram and virtual-circuit networks.

- **ROUTING ALGORITHMS**
- The main function of the network layer is routing packets from the source machine to the destination machine.
- In most networks, packets will require multiple hops to make the journey.
- The only notable exception is for broadcast networks,
- but even here routing is an issue if the source and destination are not on the same network segment.
- The algorithms that choose the routes and the data structures that they use are a major area of network layer design,
- The algorithm that manages the tables and makes the routing decisions is called the routing algorithm.
- Thereafter, data packets just follow the already established route. The latter case is sometimes called session routing because a route remains in force for an entire session.
- Routing algorithms can be grouped into two major classes: non adaptive and adaptive.
- Non adaptive algorithms do not base their routing decisions on any measurements or estimates of the current topology and traffic,

- Adaptive algorithms, in contrast, change their routing decisions to reflect changes in the topology, and sometimes changes in the traffic as well.
- These dynamic routing algorithms differ in where they get their information.
- **The Optimality Principle**
- The most efficient path based on a specific measurement metric
- This statement is known as the optimality principle (Bellman,
- 1957).
- It states that if router J is on the optimal path from router I to router K, then the optimal path from J to K also falls along the same route.
- To see this, call the part of the route from I to J r1 and the rest of the route r 2.
- If a route better than r 2 existed from J to K, it could be concatenated with r 1 to improve the route from I to K, contradicting our statement that r 1r 2 is optimal.
- As a direct consequence of the optimality principle, we can see that the set of optimal routes from all sources to a given destination form a tree rooted at the destination.
- Such a tree is called a sink tree and is illustrated in Fig. 5-6(b),
- where the distance metric is the number of hops.

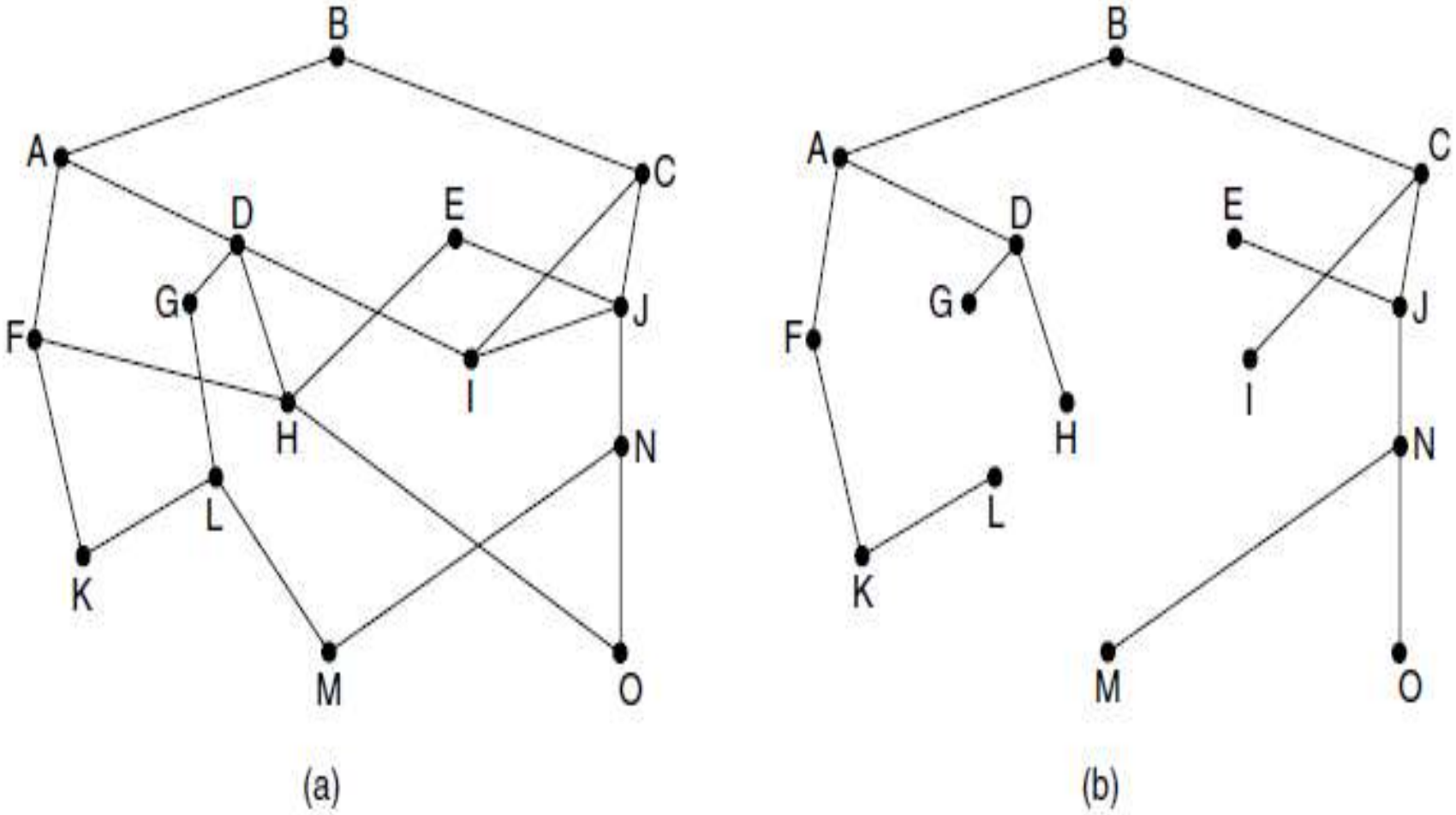
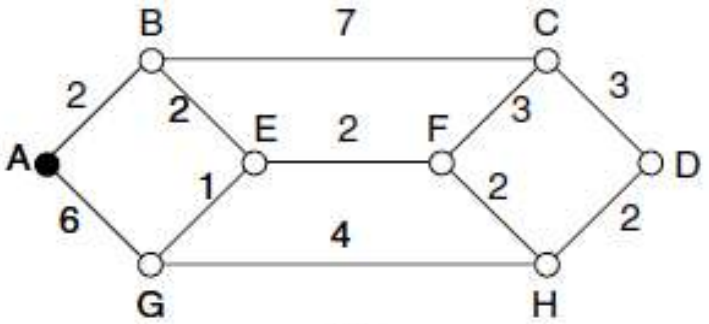
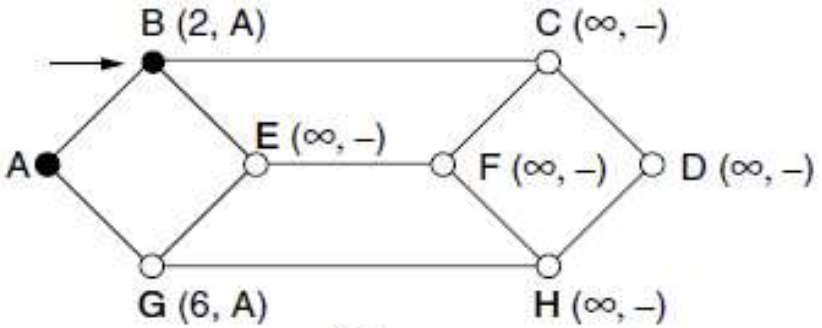
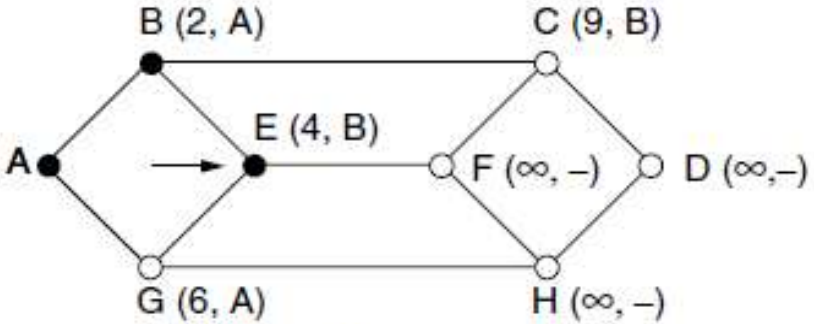


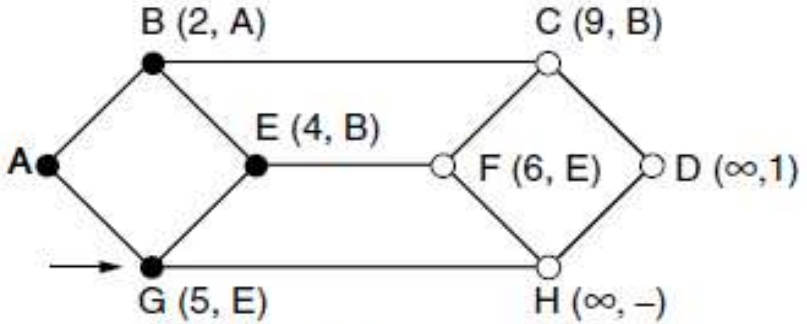
Figure 5-6. (a) A network. (b) A sink tree for router *B*.

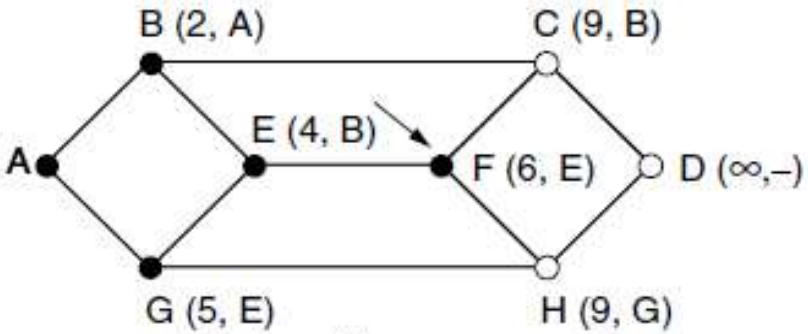
- **Shortest Path Algorithm**
- These paths are the ones that we want a distributed routing algorithm to find, even though not all routers may know all of the details of the network.
- The idea is to build a graph of the network, with each node of the graph representing a router and each edge of the graph representing a communication line, or link.
- To choose a route between a given pair of routers, the algorithm just finds the shortest path between them on the graph.
- One way of measuring path length is the number of hops. Using this metric, the paths ABC and ABE in Fig. 5-7 are equally long.
- where the weights represent, for example, distance.
- We want to find the shortest path from A to D. We start out by marking node A as permanent, indicated by a filled-in circle.
- Then we examine, in turn, each of the nodes adjacent to A (the working node), relabeling each one with the distance to A.
- Whenever a node is relabeled, we also label it with the node from which the probe was made so that we can reconstruct the final path later.
- If the network had more than one shortest path from A to D and we wanted to find all of them, we would need to remember all of the probe nodes that could reach a node with the same distance.

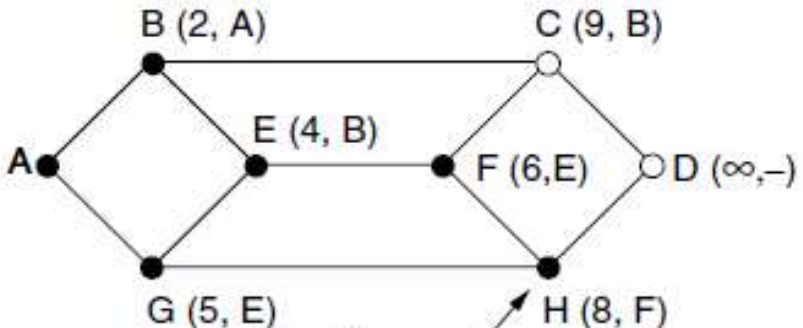

B
7
C
2
2
E
2
F
3
3
A
D
1
2
6
4
2
G
H
(a)


B (2, A)
C (∞, –)
E (∞, –)
A
F (∞, –)
D (∞, –)
G (6, A)
H (∞, –)
(b)


B (2, A)
C (9, B)
E (4, B)
A
F (∞, –)
D (∞,–)
G (6, A)
H (∞, –)
(c)


B (2, A)
C (9, B)
E (4, B)
A
F (6, E)
D (∞,1)
G (5, E)
H (∞, –)
(d)


B (2, A)
C (9, B)
E (4, B)
A
F (6, E)
D (∞,–)
G (5, E)
H (9, G)
(e)


B (2, A)
C (9, B)
E (4, B)
A
F (6,E)
D (∞,–)
G (5, E)
H (8, F)
(f)

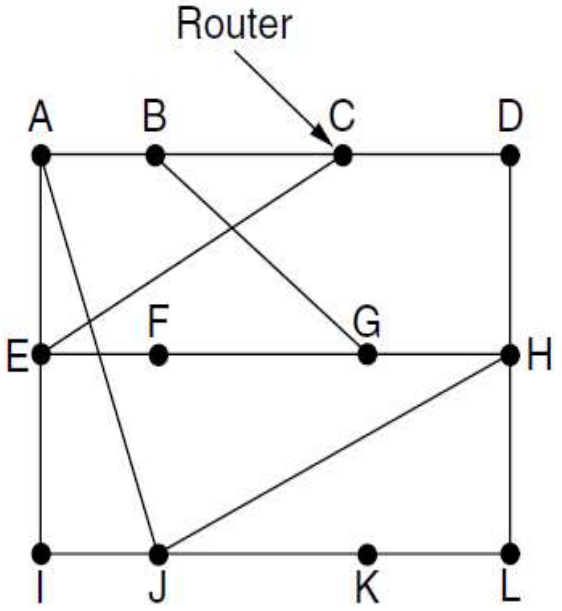
- **Flooding**
- When a routing algorithm is implemented, each router must make decisions based on local knowledge, not the complete picture of the network.
- A simple local technique is flooding, in which every incoming packet is sent out on every outgoing line except the one it arrived on.
- Flooding obviously generates vast numbers of duplicate packets, in fact, an infinite number unless some measures are taken to damp the process.
- One such measure is to have a hop counter contained in the header of each packet that is decremented at each hop, with the packet being discarded when the counter reaches zero.
- Each router then needs a list per source router telling which sequence numbers originating at that source have already been seen
- If an incoming packet is on the list, it is not flooded.

- A better technique for damming the flood is to have routers keep track of which packets have been flooded, to avoid sending them out a second time.
- One way to achieve this goal is to have the source router put a sequence number in each packet it receives from its hosts.
- **First**, it ensures that a packet is delivered to every node in the network.
- This may be wasteful if there is a single destination that needs the packet, but it is effective for broadcasting information.
- **Second,** flooding is tremendously robust. Even if large numbers of routers are blown to bits (e.g., in a military network located in a war zone), flooding will find a path if one exists, to get a packet to its destination.
- The routers only need to know their neighbors.

- **Distance Vector Routing**
- Computer networks generally use dynamic routing algorithms that are more complex than flooding, but more efficient because they find shortest paths for the
- current topology.
- Two dynamic algorithms in particular, distance vector routing and link state routing, are the most popular.
- A distance vector routing algorithm operates by having each router maintain a table (i.e., a vector) giving the best known distance to each destination and which link to use to get there.
- These tables are updated by exchanging information with the neighbors. Eventually, every router knows the best link to reach each destination.
- The distance vector routing algorithm is sometimes called by other names, most commonly the distributed **Bellman-Ford routing algorithm.**

- In distance vector routing, each router maintains a routing table indexed by, and containing one entry for each router in the network.
- This entry has two parts: the **preferred outgoing line** to use for that destination and an estimate of the distance to that destination.
- **The distance** might be measured as the number of hops or using another metric.
- Once every T msec, each router sends to each neighbor a list of its estimated delays to each destination. It also receives a similar list from each neighbor.
- Imagine that one of these tables has just come in from neighbor X, with Xi being X's estimate of how long it takes to get to router i.
- If the router knows that the delay to X is m msec, it also knows that it can reach router i via X in Xi + m msec.

- By performing this calculation for each neighbor, a router can find out which estimate seems the best and use that estimate and the corresponding link in its new routing table.
- Suppose that J has measured or estimated its delay to its neighbors, A, I, H, and K, as 8, 10, 12, and 6 msec, respectively.



(a)

To	A	I	H	K	New estimated delay from J	Line
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	–
K	24	22	22	0	6	K
L	29	33	9	9	15	K
	JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6	New routing table for J	

Vectors received from J's four neighbors

(b)

Figure 5-9. (a) A network. (b) Input from *A*, *I*, *H*, *K*, and the new routing table for *J*.

- **The Count-to-Infinity Problem**
- The settling of routes to best paths across the network is called convergence.
- Distance vector routing is useful as a simple technique by which routers can collectively compute shortest paths.
- To see how fast good news propagates, consider the five-node (linear) network of Fig.where the delay metric is the number of hops.
- Suppose A is down initially and all the other routers know this.
- In other words, they have all recorded the delay to A as infinity.
- At the time of the first exchange, B learns that its left-hand neighbor has zero delay to A. B now makes an entry in its routing table indicating that A is one hop away to the left.

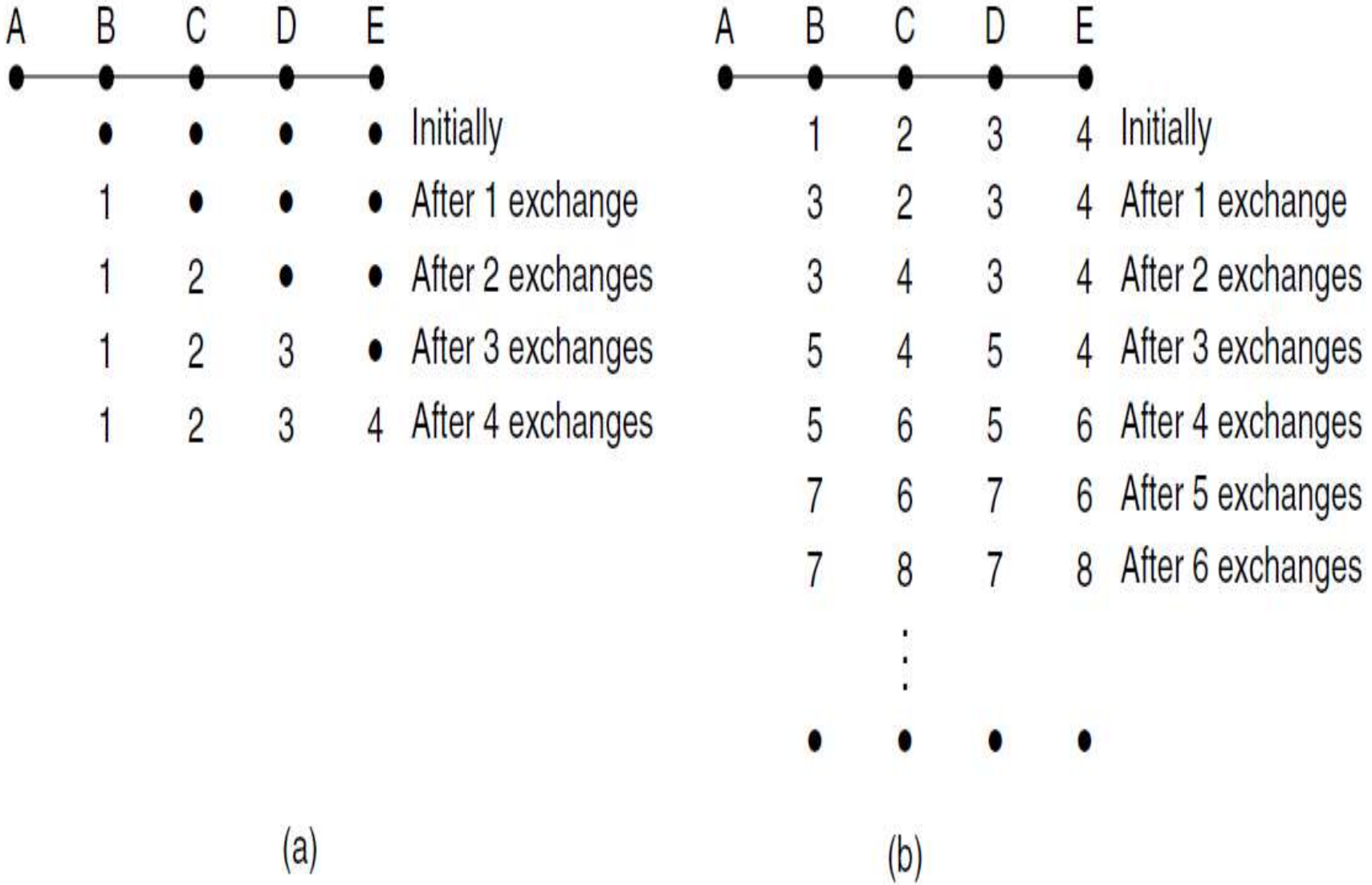


Figure 5-10. The count-to-infinity problem.

- At the first packet exchange, B does not hear anything from A. Fortunately, C have a path to A of length 2.”
- Little does B suspect that C’s path runs through B itself. For all B knows, C might have ten links all with separate paths to A of length 2.
- As a result, B thinks it can reach A via C, with a path length of 3.
- **Link State Routing**
- The primary problem that caused its demise was that the algorithm often took too long to converge after the network topology changed (due to the count-to-infinity problem).
- Consequently, it was replaced by an entirely new algorithm, now called link state routing.
- The idea behind link state routing is fairly simple and can be stated as five parts.
- Each router must do the following things to make it work:

- 1. Discover its neighbors and learn their network addresses.
- 2. Set the distance or cost metric to each of its neighbors.
- 3. Construct a packet telling all it has just learned.
- 4. Send this packet to and receive packets from all other routers.
- 5. Compute the shortest path to every other router.
- **Learning about the Neighbo**rs
- When a router is booted, its first task is to learn who its neighbors are.
- It accomplishes this goal by sending a special HELLO packet on each point-to-point line.
- The router on the other end is expected to send back a reply giving its name.
- **Setting Link Costs**
- The link state routing algorithm requires each link to have a distance or cost metric for finding shortest paths

- The cost to reach neighbors can be set automatically,
- or configured by the network operator.
- A common choice is to make the cost inversely proportional to the bandwidth of the link.

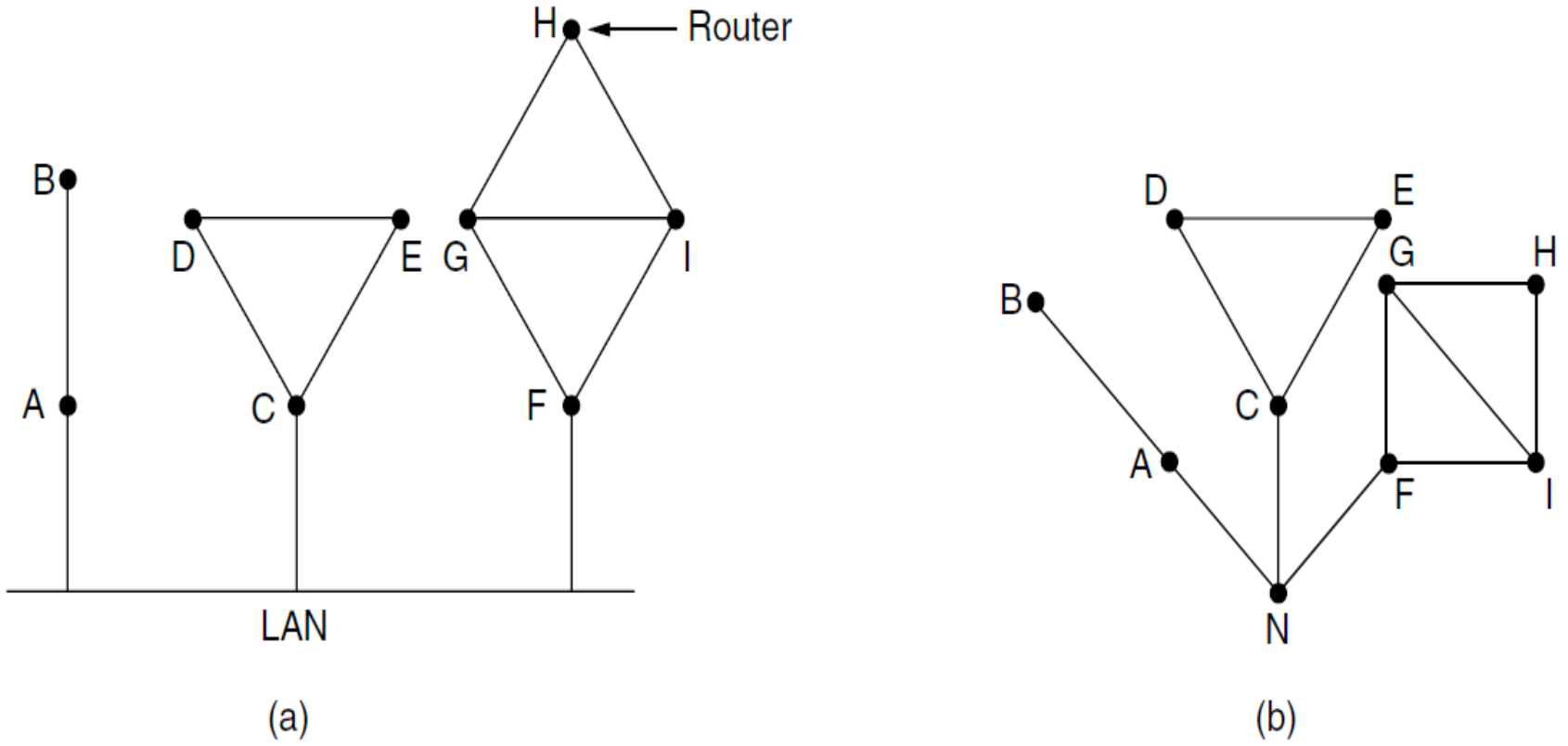


Figure 5-11. (a) Nine routers and a broadcast LAN. (b) A graph model of (a).

- **Building Link State Packets**
- Once the information needed for the exchange has been collected, the next step is for each router to build a packet containing all the data.
- The packet starts with the identity of the sender, followed by a sequence number and age (to be described
- later) and a list of neighbors.

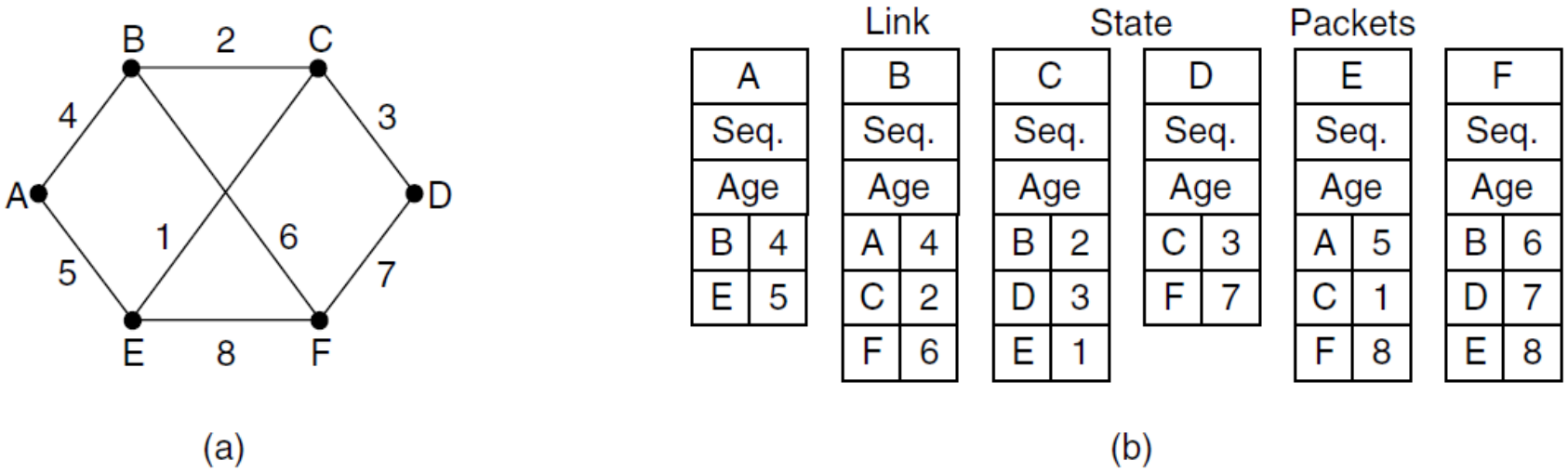
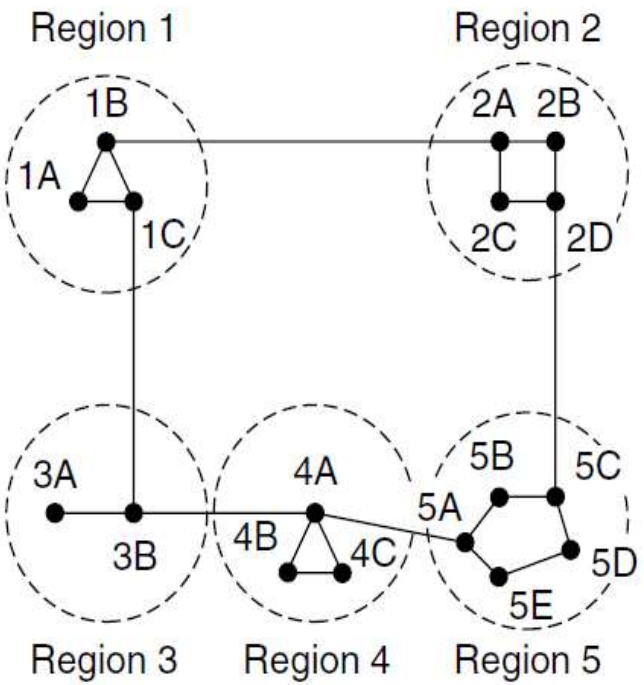


Figure 5-12. (a) A network. (b) The link state packets for this network.

- **Hierarchical Routing**
- As networks grow in size, the router routing tables grow proportionally.
- Not only is router memory consumed by ever-increasing tables, but more CPU time is needed to scan them and more bandwidth is needed to send status reports about them.
- When hierarchical routing is used, the routers are divided into what we will call **regions**.
- Each router knows all the details about how to route packets to destinations within its own region but knows nothing about the internal structure of other regions.
- Figure gives a quantitative example of routing in a two-level hierarchy with five regions.
- For example, consider a network with 720 routers.
- If there is no hierarchy, each router needs 720 routing table entries.
- If the network is partitioned into 24 regions of 30 routers each, each router needs 30 local entries plus 23 remote entries for a total of 53 entries.

- Each router stores routing entries for all routers in its own local region plus one entry for each of the other regions in the network.



(a)

Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

Figure 5-14. Hierarchical routing.

- **Broadcast Routing**
- Sending a packet to all destinations simultaneously is
- called broadcasting. Various methods have bee proposed for doing it.
- One broadcasting method that requires no special features from the network is for the source to simply send a distinct packet to each destination.
- An improvement is **multidestination** routing, in which each packet contains either a list of destinations or a bit map indicating the desired destinations.
- When a packet arrives at a router, the router checks all the destinations to determine the set of output lines that will be needed.
- The router generates a new copy of the packet for each output line to be used and includes in each packet only those destinations that are to use the line

- The network bandwidth is therefore used more efficiently.
- scheme still requires the source to know all the destinations.
- much work for a router to determine where to send one multidestination packet as it is for multiple distinct packets.
- The idea for **reverse path forwarding**
- When a broadcast packet arrives at a router, the router checks to see if the packet arrived on the link that is
- normally used for sending packets toward the source of the broadcast.
- If so, there is an excellent chance that the broadcast packet itself followed the best route from the router and is therefore the first copy to arrive at the router.
- This being the case, the router forwards copies of it onto all links except the one it arrived on.

- The principal advantage of reverse path forwarding is that it is efficient while being easy to implement.
- It sends the broadcast packet over each link only once in each direction, just as in flooding, yet it requires only that routers know how to reach all destinations, without needing to remember sequence numbers (or use
- other mechanisms to stop the flood) or list all destinations in the packet.
- **A spanning tree** is a subset of the network that includes all the routers but contains no loops. Sink trees are spanning
- trees.
- If each router knows which of its lines belong to the spanning tree, it can copy an incoming broadcast packet onto all the spanning tree lines except the one it arrived on.
- This method makes excellent use of bandwidth, generating the absolute minimum number of packets necessary to do the job.

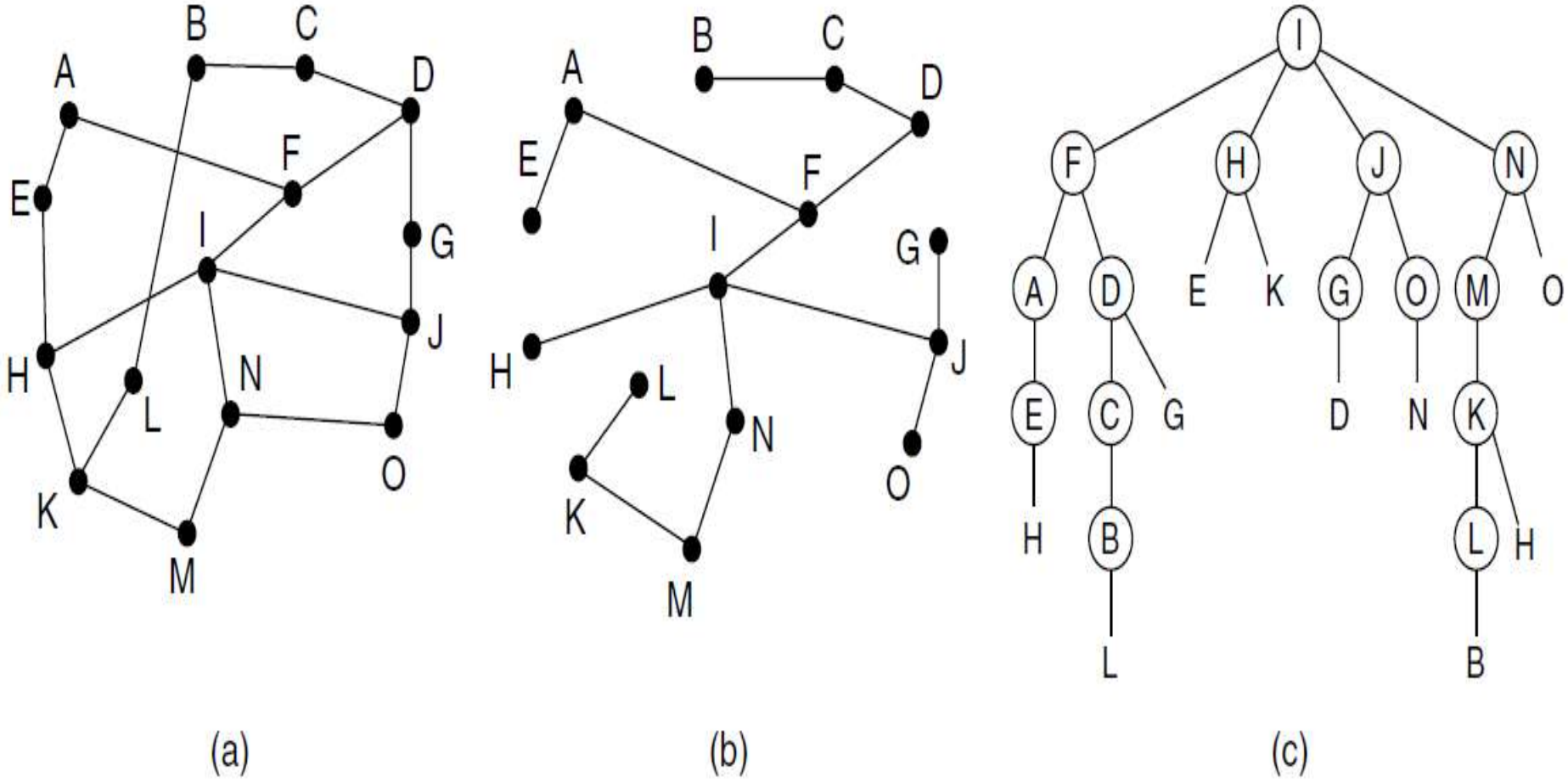


Figure 5-15. Reverse path forwarding. (a) A network. (b) A sink tree. (c) The tree built by reverse path forwarding.

- **Multicast Routing**
- Sending a message to such a group is called multicasting, and the routing algorithm used is called multicast routing.
- All multicasting schemes require some way to create and destroy groups and to identify which routers are members of a group.
- Multicast routing schemes build on the broadcast routing schemes we have already studied, sending packets along spanning trees to deliver the packets to the
- members of the group while making efficient use of bandwidth.
- However, the best spanning tree to use depends on whether the group is dense, with receivers scattered over most of the network, or sparse.
- The result is an efficient multicast spanning tree.

- consider the two groups, 1 and 2, in the network shown in Fig. 5-16(a). Some routers are attached to hosts that belong to one or both of these groups, as indicated in the figure.
- A spanning tree for the leftmost router is shown in Fig. 5-16(b). This tree can be used for broadcast but is overkill for multicast, as can be seen from the two pruned versions that are shown next.
- In Fig. 5-16(c), all the links that do not lead to hosts that are members of group 1 have been removed.
- Packets are forwarded only along this spanning tree, which is more efficient than the broadcast tree because there are 7 links instead of 10.
- Fig. 5-16(d) shows the multicast spanning tree after pruning for group 2. It is efficient too, with only five links this time. It also shows that different multicast groups have different spanning trees.

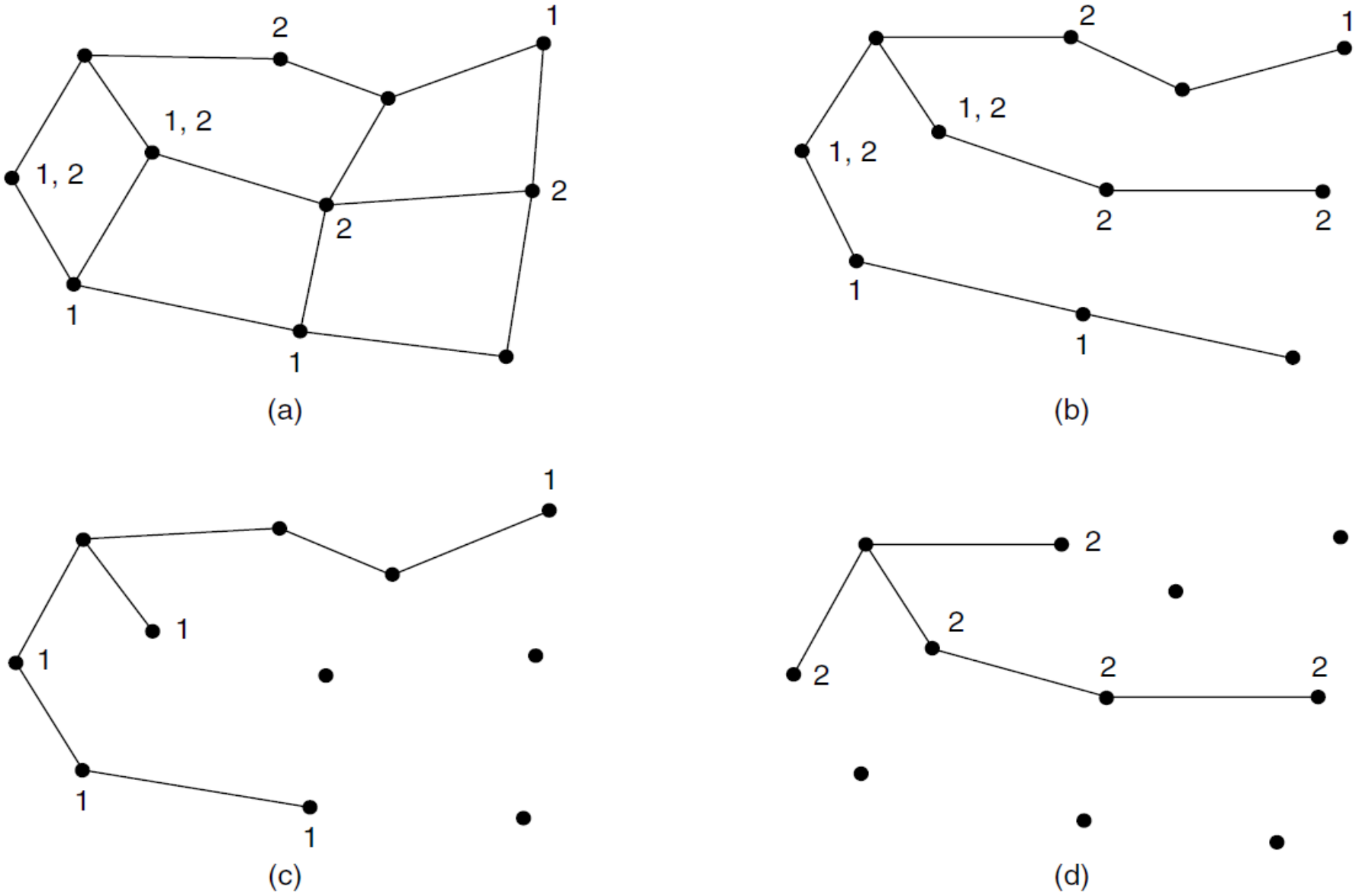


Figure 5-16. (a) A network. (b) A spanning tree for the leftmost router. (c) A multicast tree for group 1. (d) A multicast tree for group 2.

- An alternative design uses **core-based trees** to compute a single spanning tree for the group All of the routers agree on a root (called the core or rendezvous point) and build the tree by sending a packet from each member to the root.
- The tree is the union of the paths traced by these packets. Fig. 5-17(a) shows a core-based tree for group 1.
- To send to this group, a sender sends a packet to the core.
- When the packet reaches the core, it is forwarded down
- the tree.
- This is shown in Fig. 5-17(b) for the sender on the righthand side of the network.
- Having a shared tree is not optimal for all sources.
- the packet from the sender on the righthand side reaches the top-right group member via the core in three hops, instead of directly.
- The inefficiency depends on where the core and senders are located, but often it is reasonable when the core is in the middle of the senders.

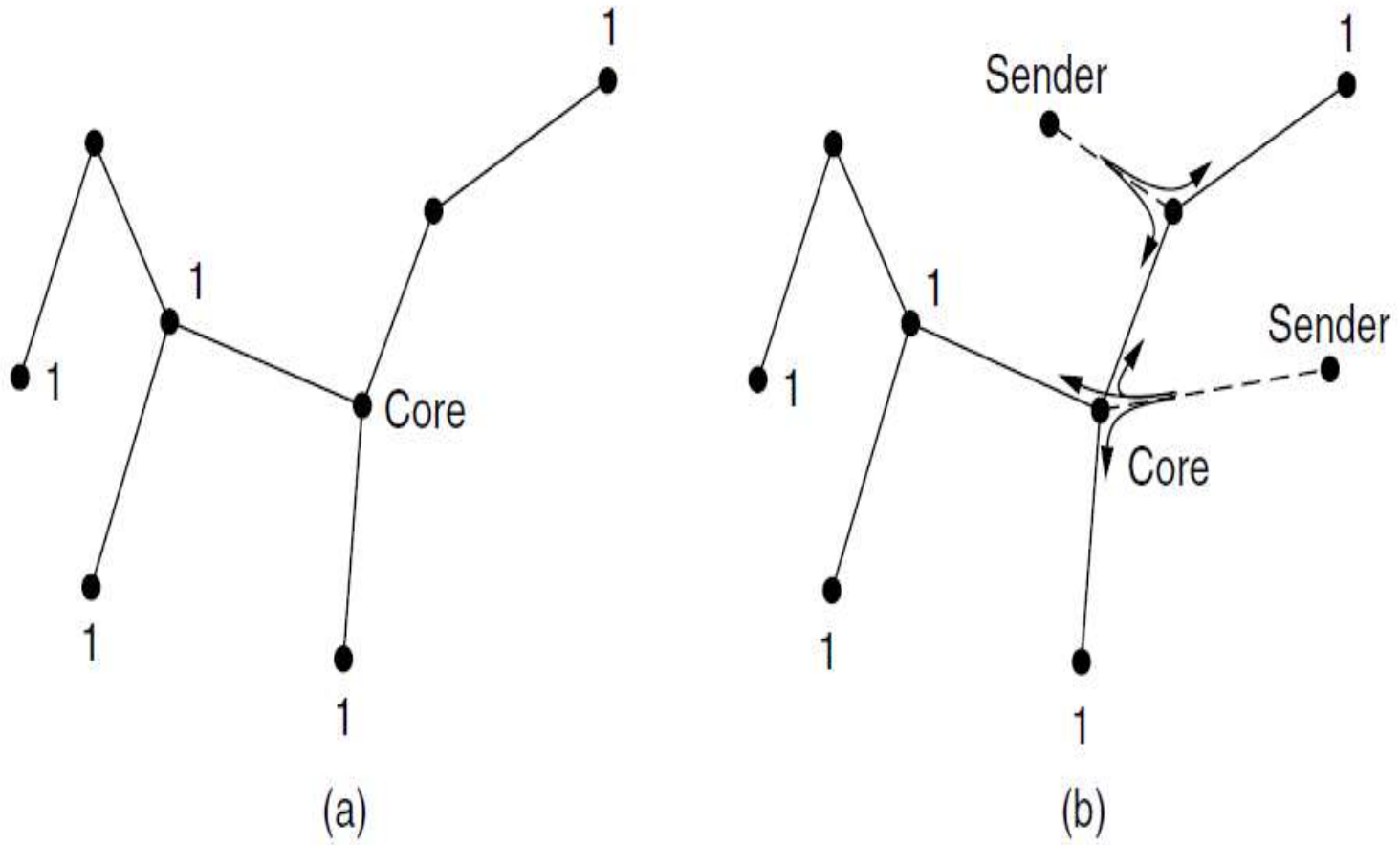
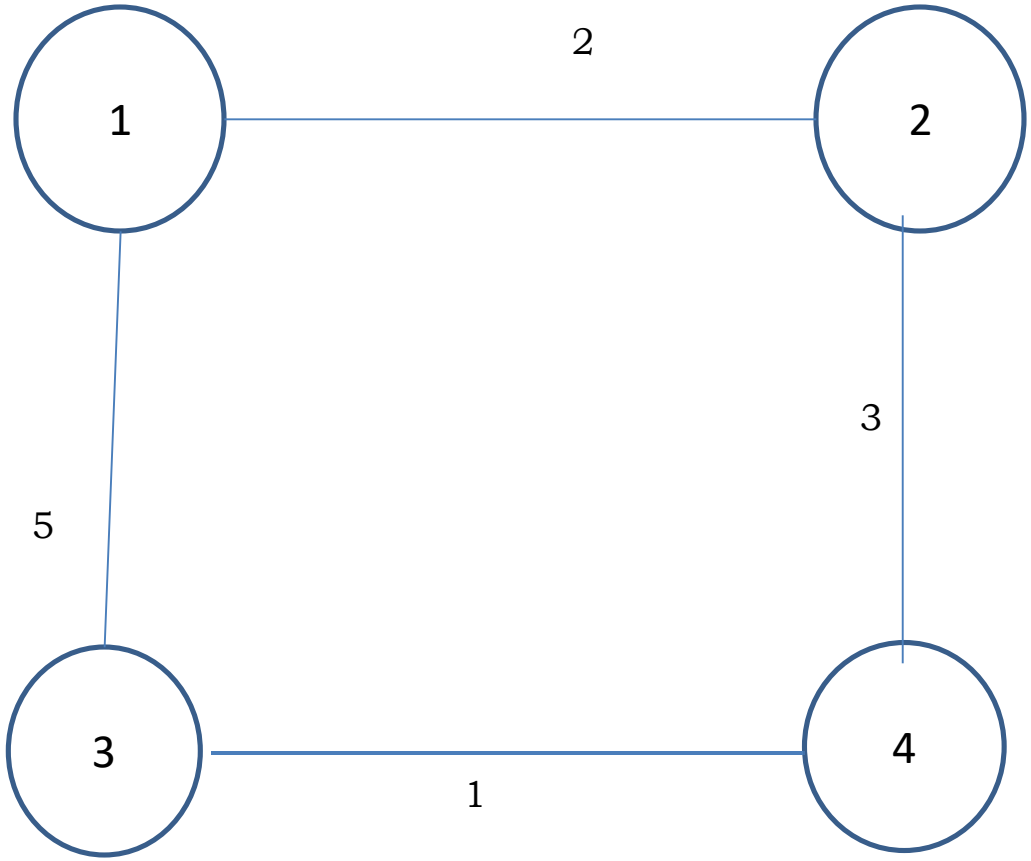


Figure 5-17. (a) Core-based tree for group 1. (b) Sending to group 1.


1
2
2
3
5
3
4
1